Shopping and Budget Advisor

CSIT6000T Course Project Proposal

Hechuan ZHANG, Bolin CHEN, Jieming CHEN

1 Motivation and task

A low listing price can become a poor choice once delivery charges, missing accessories, or unsuitable specifications are considered. Reviews may refer to different versions, repeat claims, or disagree about important attributes. For a basket, substitutions also change whether the purchase meets the budget. Buyers must check evidence across sources and decide which tradeoffs fit their needs.

We propose a shopping and budget advisor for any user group and product category. Individuals, households, and teams need recommendations that reflect intended use and available funds. Users will describe essential requirements and possible compromises. The advisor will compare alternatives, explain tradeoffs with source evidence, and revise plans as priorities change, leaving the purchase decision with the user.

We will handle one request at a time and return ranked products or baskets with itemized costs, links, tradeoffs, and remaining funds. Users may set a purchase cap or supply a periodic budget, spending, and a reserve. When both limits are given, available funds are the smaller of the cap and the balance after spending and reserves. Missing essential specifications or costs trigger clarification.

2 Proposed system

Six layers (Figure 1) extend the repository's TACC deployment and inference client with shopping, knowledge, and evaluation capabilities. Training and new quantization depend on available resources.

01 Application	**Browser frontend** Goals, budgets, revisions	**Python backend** Sessions and API bridge	**Bounded agent** Retrieve, compare, validate
02 Knowledge	**SerpApi connector** Shopping and Product APIs	**Evidence store** SQLite, offers, raw JSON	**Evidence retrieval** Filters, FTS5, source IDs
03 Post-training	**Demonstrations** Development traces only	**SFT; optional DPO** LoRA / QLoRA adapters	**Model export** Checkpoint and tokenizer
04 Inference	**Calibrate and quantize** Same checkpoint; INT8 / INT4	**vLLM service** TP=2; 4K; two sequences	**Serving acceleration** Batching, cache, execution
05 Evaluation	**Task and budget checks** Success, constraints, amounts	**Grounding and retrieval** Ragas, Recall@5, MRR@5	**System measurements** User latency, TTFT, VRAM
06 TACC	**SDK and Slurm jobs** GPU service and CPU worker	**Runtime and storage** Course environment, USERDIR	**File bridge** Request IDs and deadlines

Figure 1: Planned architecture. SerpApi feeds the knowledge base; budget tools validate plans; post-training targets agent behavior.

01 Agent Application Layer. A browser will collect goals and budgets and support progress, sources, and revisions. The backend will maintain sessions; a bounded agent will clarify, retrieve, compare, and validate. Schema-checked Python tools will check requirements, quantities, known taxes/shipping, and exact totals.

02 Knowledge Layer. Backend SerpApi Shopping and Product calls will acquire listings and available offers, specifications, and reviews [1]. Sanitized JSON and persistent SQLite records will preserve variants, sources, currency, and timestamps. Deduplication will retain conflicts; filters and FTS5 will retrieve cited evidence. Live refreshes are API-budgeted; evaluation uses immutable snapshots.

03 Post-Training Layer. Development demonstrations will teach clarification, evidence selection, tool calls, and recovery. SFT with LoRA/QLoRA requires a trainable checkpoint; DPO is optional. Held-out cases and the unseen category are excluded from training and calibration. Exports retain tokenizer and model configuration.

04 Quantization and Inference. Supported INT8/INT4 variants will share a checkpoint and generation settings. The initial vLLM service uses tensor parallelism of two, 4K context, and at most two sequences. Context selection, batching, caching, and execution modes will be compared for quality, latency, and total GPU memory.

05 Evaluation Layer. A harness will replay frozen requests and log model/snapshot IDs, prompts, seeds, observations, tokens, timing, and errors. Rules and human review will distinguish retrieval, tool-use, arithmetic, and serving failures.

06 TACC Infrastructure Layer. Tcloud/Slurm will run a GPU service on two observed 24 GiB GPUs and a separate CPU worker. An SDK file bridge keeps model calls inside TACC; public API calls and credentials stay on the backend. USERDIR persistence, deadlines, and job states support one-hour allocations.

3 Evaluation plan

Representative tasks and quality. We will prepare 10 development and 40 held-out cases across five categories; eight cases test a fifth category added through schemas and data after freezing the agent, prompts, and tools. Cases cover products, baskets, unit-price differences, budget boundaries, substitutions, revisions, and incomplete evidence. Independent enumeration of the frozen catalog will establish feasibility and exact costs. Task success rate (TSR) requires a feasible, correctly priced plan, a correct catalog-limited no-plan report, or a necessary clarification; unnecessary abstention is failure. We target zero budget violations and arithmetic errors and report two reviewers' preference-fit scores (1–5). A shared rubric will score specification fit, justified tradeoffs, and completeness. Unknown shipping or taxes must remain explicit; an unsupported estimate cannot establish affordability. Revision cases will check that unchanged requirements persist.

Knowledge base and retrieval. With about 100 products and 300 review excerpts where available, we will audit field coverage, duplicates, source agreement, and price age. Reviewers will label relevant evidence for Recall@5 and MRR@5. Ragas Context Precision/Recall, Faithfulness, and Factual Correctness will supplement citation support/coverage checks [2]. Faithfulness measures consistency with supplied evidence, not whether prices remain current. Retrieval comparisons will share structured candidates, prices, the model, and budget tools; missing/stale facts will be distinguished from retrieval errors. Product variants will receive stable IDs; near-duplicate reviews will stay within the same data split. References will be labeled before viewing system outputs, with disagreements resolved against archived evidence.

Baselines and efficiency. Table 1 specifies controlled baselines and ablations. Base/SFT comparisons share a base model; precision comparisons share a trained checkpoint. We will vary sequential/two-way analysis, filtered/unfiltered context, and serving settings one factor at a time. Each held-out case will run three times per core configuration, interleaved on the same endpoint, with case-paired uncertainty. The target is 20% lower median latency or 30% fewer input tokens versus the sequential, unfiltered agent, with at most a five-percentage-point success loss. Timeouts, malformed outputs, stale offers, and missing fees will be injected, with at least one failure trace analyzed. Development will proceed from SDK transport and data ingestion to retrieval, deterministic budget tools, and the complete agent. Baseline evaluation precedes adapter training and quantization; each stage must pass the same frozen task checks.

Method	Configuration	TSR % ↑	Faith. ↑	BVR % ↓	p95 s ↓
B0 Single call	No extra text; no budget tool	n/m	n/m	n/m	n/m
B1 Keyword RAG	FTS5; single call; no budget tool	n/m	n/m	n/m	n/m
B2 Structured RAG	Filtered FTS5; single call; no budget tool	n/m	n/m	n/m	n/m
B3 Full advisor	B2 + bounded loop + budget tool	n/m	n/m	n/m	n/m
A1 Without loop	B3; one-pass plan with the same tools	n/m	n/m	n/m	n/m
A2 Without validator	B3; budget tool disabled	n/m	n/m	n/m	n/m

Table 1: Planned baseline and ablation results; n/m = not measured. Faithfulness is on a 0–1 scale; p95 is full-request latency. Budget violation rate (BVR) uses delivered plans as its denominator. All methods share structured candidates and prices. A1 isolates the loop; A2 isolates the validator.

Pipeline stage	Measures and measurement boundary
Acquisition / KB	Required-field coverage, source agreement, duplicates, price age; API calls, refresh/cache state, errors, and acquisition latency.
Retrieval	Recall@5 and MRR@5 from evidence IDs; Context Precision/Recall from ranked evidence and reference facts [2].
Answer / agent	Faithfulness, Factual Correctness, citation coverage, preference fit; schema validity and Tool Call Accuracy [2,3].
Budget / task	TSR, BVR, hard-constraint violations, and exact amount error from independent feasibility and arithmetic checks.
Model serving	TTFT, TPOT, ITL, request/output-token throughput, SLO goodput, and peak total GPU memory [4].
Whole request	User p50/p95 latency, completion/failure rate, tokens, API/LLM calls, and cost per successful task; trace queue/bridge/tool spans.

Table 2: Full-pipeline measurement. TTFT = time to first token; TPOT = time per output token; ITL = inter-token latency; SLO = service level objective. The current blocking client needs streaming instrumentation to measure token timing.

Reporting controls. Freeze snapshots for quality comparisons and report live acquisition separately. Warm up serving and separate cold starts/cache states. Model TTFT excludes earlier acquisition and bridge waits. Include failures, API retries, and unsuccessful tasks in cost accounting; freeze SLO thresholds on development cases. Ragas judges will be versioned and checked against human labels; deterministic budget checks remain authoritative. We will report paired bootstrap intervals over cases, preserving repeated runs within each case. BVR is undefined when no plans are delivered. Throughput gains will be interpreted alongside successful tasks per unit time. Reports will archive model and runtime configurations, case definitions, snapshot manifests, evaluation scripts, and representative traces for reproducibility.

References. [1] SerpApi Shopping / Product APIs. [2] Ragas RAG metrics. [3] Ragas tool metrics. [4] vLLM benchmarks.